Assignment 2 Report

All results are generated by using Python with pandas and numpy imported.

The data used are “tickers.csv”, “univ_h.csv”, and “adjusted.csv”.

Part A

Steps:

1. Generate industry code for each stock. Drop stocks without GISC code or with GISC code equal to “#N/A Invalid Security”. Retain GOOGL and discard GOOG. Retain NWSA and discard NWS because NWSA has higher liquidity.
2. Generate log return for each stock on each day, which is also the daily return for each stock. **Remove the first day of 2004.** Fill the NA with 0.
3. Generate prior 5-day log return for each stock on each day. **Remove the first 5 days of 2004.** Fill the NA with 0.
4. Generate the volatility using the prior 21 days of daily returns with **minimal periods of rolling equal to 5**. Remove the first 4 days of 2004 so that the sample size is aligned with that of prior 5-day log return data. Set volatility equal to 0.003 if it is less than 0.003.
5. Normalize the prior 5-day log return with volatility and get df_nml. **Ensure column names contain no special characters by using df_nml.columns.str.strip()**
6. Generate each year’s ticker list that belongs to the universe.
7. Create a mask with size equal to df_nml. For each year, only keep values that belongs to stocks in universe and set other values equal to np.nan. **We should filter the dataframe with tickers in universe before industry demeaning. Convert the dataframe to long format using “stack()”.**
8. Merge the new df_nml and industry code on ticker. Calculate simple average of prior 5-day return for each industry and date using “groupby()”. If the stock doesn’t have industry code or prior 5-day return data, it is excluded in the calculation. **Subtract industry component from variable.**
9. Using the same steps to filter the daily return data with tickers included in the universe. Convert the daily return data to long format and **demean daily return data by industry mean.**
10. Merge the new df_nml and daily return data, sort values based on date and ticker, and get df_final. Shift the daily return data up by one row for each ticker and get next day return data. **We must first conduct industry demean and then shift the daily return so that the returns are demeaned by correct industry.**
11. Calculate the beta according to the formula. **Skip beta calculation if return or variable is missing. This means when t+1 is the beginning of year y+1 and t is the end of year y, we will only consider stocks with both variable and return data.**
12. Calculate mean, standard deviation, and t-stat of beta for each year.

Result:

	year	mean	std	count	t_stat
0	2004	-0.000136	0.000624	247	-3.427817
1	2005	-0.000133	0.000549	252	-3.845856
2	2006	-0.000160	0.000576	251	-4.401218
3	2007	-0.000081	0.000901	251	-1.422194
4	2008	-0.000497	0.003370	253	-2.348262
5	2009	-0.000137	0.002006	252	-1.086134
6	2010	-0.000057	0.000763	252	-1.195891
7	2011	-0.000176	0.001293	252	-2.164281
8	2012	0.000022	0.000676	250	0.511471
9	2013	-0.000065	0.000562	252	-1.842169
10	2014	-0.000025	0.000642	252	-0.624152
11	2015	-0.000005	0.000893	252	-0.097222
12	2016	-0.000070	0.001048	252	-1.055238
13	2017	-0.000043	0.000593	251	-1.155244
14	2018	0.000028	0.000935	251	0.472556
15	2019	0.000090	0.001160	252	1.227006
16	2020	-0.000375	0.003804	253	-1.568538
17	2021	0.000015	0.001479	252	0.161380
18	2022	-0.000013	0.001790	251	-0.119097
19	2023	-0.000173	0.001740	250	-1.575897
20	2024	-0.000104	0.000878	252	-1.883139
21	2025	0.000003	0.001411	249	0.037566

Comment:

The market is mean-reverting according to the prior 5-day return because most of t-stats are negative.

The magnitude of t-stats is higher in the early years (2004-2011) but lower in the recent years, indicating that the mean-reverting factor is losing its effect in the recent years.

Some positive t-stats may indicate special market environment of the corresponding years and the market is not mean-reverting.

Part B

Steps:

1. For each year y, calculate the average β from year max(2004, y-5) to year y−1
2. Merge df_final and beta bar data. Drop the years without beta bar. Calculate the expected return using beta bar and variable. **Drop those observations without expected return.**
3. Rank the ticker by expected return for each date. Create signal (1/0/-1) based on the rank. Convert log return to simple return.
4. Shift the signal down one row for each ticker because today's signals correspond to the next day's positions. Drop rows with NA positions.
5. Generate the portfolio return at each time step by using "groupby('date').apply()" with a function using mask to calculate each day's portfolio return. When calculating the portfolio return, the full return without subtracting market return is used.
6. Calculate total annual return, annual return volatility and sharpe ratio of the portfolio.

Result:

If the portfolio return is log return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.190455	0.041244	4.617770
1	2007	0.088001	0.065845	1.336490
2	2008	0.391532	0.174598	2.242474
3	2009	0.109670	0.112984	0.970663
4	2010	0.055799	0.045738	1.219981
5	2011	0.130569	0.062524	2.088287
6	2012	0.024490	0.048039	0.509802
7	2013	0.040373	0.037840	1.066937
8	2014	0.033948	0.042939	0.790602
9	2015	0.018476	0.054238	0.340644
10	2016	0.029181	0.065333	0.446653
11	2017	0.071588	0.042162	1.697940
12	2018	-0.052182	0.052882	-0.986755
13	2019	-0.045562	0.069855	-0.652238
14	2020	0.305517	0.227956	1.340246
15	2021	0.030800	0.097973	0.314375
16	2022	0.091288	0.105011	0.869325
17	2023	0.086310	0.101511	0.850247
18	2024	0.129462	0.065771	1.968373

If the portfolio return is simple return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.193411	0.041480	4.662778
1	2007	0.095204	0.065689	1.449316
2	2008	0.489273	0.177393	2.758131
3	2009	0.116485	0.113923	1.022486
4	2010	0.057587	0.046140	1.248106
5	2011	0.144049	0.062353	2.310227
6	2012	0.031025	0.048323	0.642034
7	2013	0.044356	0.037989	1.167620
8	2014	0.037012	0.042798	0.864800
9	2015	0.026472	0.054244	0.488024
10	2016	0.033925	0.065404	0.518700
11	2017	0.077183	0.042068	1.834703
12	2018	-0.045347	0.052663	-0.861086
13	2019	-0.032402	0.069320	-0.467427
14	2020	0.359413	0.223838	1.605686
15	2021	0.031828	0.098009	0.324747
16	2022	0.110289	0.104589	1.054503
17	2023	0.190540	0.119254	1.597760
18	2024	0.128885	0.065405	1.970569

Comment:
According to the sharpe ratio, the best year of the strategy is 2006 and the worst year of the strategy is 2018.

Part C
Step:

1. Create a “trade” column to record the position change for each stock using today’s signal and today’s position. if a position changes from short to long, the trade is recorded as 2.
2. Update the function with trading cost, which is equal to (cost per trade * position

change). Generate portfolio return using the same steps as previous.

Result:

If the portfolio return is log return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.015117	0.040844	0.370118
1	2007	-0.084480	0.065430	-1.291155
2	2008	0.212849	0.174141	1.222284
3	2009	-0.063418	0.112534	-0.563547
4	2010	-0.115639	0.045462	-2.543630
5	2011	-0.044533	0.062145	-0.716598
6	2012	-0.148896	0.047615	-3.127101
7	2013	-0.132584	0.037505	-3.535121
8	2014	-0.139809	0.042672	-3.276374
9	2015	-0.153106	0.053791	-2.846284
10	2016	-0.141978	0.064888	-2.188044
11	2017	-0.098731	0.041832	-2.360203
12	2018	-0.220858	0.052333	-4.220228
13	2019	-0.216470	0.069259	-3.125514
14	2020	0.127672	0.227349	0.561568
15	2021	-0.139785	0.097511	-1.433526
16	2022	-0.078906	0.104451	-0.755435
17	2023	-0.081888	0.101090	-0.810053
18	2024	-0.042569	0.065441	-0.650487

If the portfolio return is simple return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.018073	0.041080	0.439945
1	2007	-0.077277	0.065279	-1.183796
2	2008	0.310591	0.176938	1.755364
3	2009	-0.056603	0.113471	-0.498836
4	2010	-0.113851	0.045862	-2.482475
5	2011	-0.031053	0.061972	-0.501077
6	2012	-0.142361	0.047900	-2.972048
7	2013	-0.128601	0.037650	-3.415696
8	2014	-0.136745	0.042532	-3.215097
9	2015	-0.145109	0.053802	-2.697080
10	2016	-0.137235	0.064962	-2.112547
11	2017	-0.093136	0.041735	-2.231594
12	2018	-0.214023	0.052112	-4.106996
13	2019	-0.203310	0.068721	-2.958471
14	2020	0.181569	0.223230	0.813371
15	2021	-0.138757	0.097550	-1.422418
16	2022	-0.059905	0.104028	-0.575852
17	2023	0.022342	0.118902	0.187902
18	2024	-0.043146	0.065071	-0.663062

Comment:

Compared to the result without considering trading cost, this result has lower total annual return and sharpe ratio.

Part D

Steps:

1. Generate w0 for each date. Calculate normalized expected return.
2. Set gamma equal to 1.5. Create a date list and a previous position list with initial value equal to the first day's signal.
3. For each date, generate a new dataframe with position appended, and calculate x and y array using the normalized expected return and hurdle. Get the new position according to the value of x array and y array.
4. Update the position and loop to the next day.
5. Create "trade" column to record the position change.
6. Update the function with trading cost. Generate portfolio return using the same steps as previous.

Result:

Gamma = 1.5

If the portfolio return is log return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.026770	0.041676	0.642342
1	2007	-0.012744	0.061647	-0.206724
2	2008	0.262167	0.168392	1.556891
3	2009	0.052139	0.113702	0.458562
4	2010	-0.003282	0.043683	-0.075135
5	2011	0.021072	0.057791	0.364624
6	2012	-0.103587	0.046048	-2.249542
7	2013	0.008141	0.037153	0.219118
8	2014	-0.028709	0.045678	-0.628501
9	2015	-0.081233	0.051952	-1.563614
10	2016	0.028924	0.060224	0.480277
11	2017	-0.056292	0.042344	-1.329394
12	2018	-0.114263	0.056022	-2.039627
13	2019	-0.038581	0.062998	-0.612419
14	2020	0.231309	0.220518	1.048934
15	2021	-0.045995	0.093262	-0.493183
16	2022	-0.013972	0.095117	-0.146892
17	2023	-0.022017	0.092660	-0.237614
18	2024	0.004529	0.062022	0.073021

If the portfolio return is simple return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.030421	0.041880	0.726398
1	2007	0.000346	0.061484	0.005623
2	2008	0.348864	0.165716	2.105184
3	2009	0.070956	0.114687	0.618694
4	2010	-0.000343	0.044009	-0.007794
5	2011	0.033495	0.057787	0.579630
6	2012	-0.097746	0.046115	-2.119642
7	2013	0.012386	0.037482	0.330452
8	2014	-0.022610	0.045496	-0.496973
9	2015	-0.073545	0.052006	-1.414161
10	2016	0.036918	0.059877	0.616562
11	2017	-0.050411	0.042249	-1.193186
12	2018	-0.104321	0.055760	-1.870880
13	2019	-0.024285	0.062915	-0.386002
14	2020	0.287279	0.217969	1.317979
15	2021	-0.044131	0.093222	-0.473396
16	2022	0.004998	0.095313	0.052434
17	2023	0.125765	0.114274	1.100562
18	2024	0.007767	0.061683	0.125923

For different hurdle rate:
Gamma = 1
If the portfolio return is log return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.024676	0.042893	0.575287
1	2007	-0.006320	0.063554	-0.099446
2	2008	0.253887	0.169837	1.494880
3	2009	-0.007131	0.120290	-0.059280
4	2010	-0.015711	0.044432	-0.353589
5	2011	0.005623	0.061251	0.091807
6	2012	-0.083658	0.046444	-1.801285
7	2013	-0.018278	0.037594	-0.486192
8	2014	-0.061854	0.044675	-1.384544
9	2015	-0.103473	0.054844	-1.886685
10	2016	0.013932	0.062619	0.222480
11	2017	-0.066467	0.044004	-1.510474
12	2018	-0.157152	0.055962	-2.808190
13	2019	-0.087133	0.066599	-1.308323
14	2020	0.212025	0.220131	0.963177
15	2021	-0.081936	0.093646	-0.874957
16	2022	-0.023608	0.097941	-0.241040
17	2023	-0.014820	0.096659	-0.153321
18	2024	-0.010162	0.063058	-0.161148

If the portfolio return is simple return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.028573	0.043017	0.664211
1	2007	0.006358	0.063052	0.100833
2	2008	0.348394	0.170045	2.048834
3	2009	0.007776	0.120155	0.064719
4	2010	-0.012746	0.044924	-0.283734
5	2011	0.018390	0.061251	0.300245
6	2012	-0.078946	0.046585	-1.694652
7	2013	-0.013756	0.037841	-0.363531
8	2014	-0.056550	0.044556	-1.269174
9	2015	-0.097374	0.055102	-1.767153
10	2016	0.018744	0.062630	0.299283
11	2017	-0.061014	0.043968	-1.387699
12	2018	-0.148912	0.055867	-2.665468
13	2019	-0.073508	0.066593	-1.103833
14	2020	0.264532	0.218341	1.211557
15	2021	-0.080620	0.093582	-0.861490
16	2022	-0.005252	0.097855	-0.053672
17	2023	0.123926	0.115767	1.070482
18	2024	-0.009284	0.062683	-0.148113

Gamma = 2

If the portfolio return is log return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.039599	0.041696	0.949712
1	2007	-0.029032	0.058096	-0.499726
2	2008	0.214133	0.163864	1.306774
3	2009	0.095247	0.107688	0.884473
4	2010	0.012209	0.043195	0.282638
5	2011	-0.010182	0.061013	-0.166875
6	2012	-0.126863	0.047510	-2.670259
7	2013	0.016685	0.037088	0.449880
8	2014	-0.039215	0.046456	-0.844136
9	2015	-0.094673	0.054220	-1.746085
10	2016	0.053764	0.057893	0.928673
11	2017	-0.034583	0.043132	-0.801805
12	2018	-0.097141	0.056128	-1.730696
13	2019	-0.012603	0.063012	-0.200006
14	2020	0.237469	0.208137	1.140924
15	2021	-0.033398	0.087202	-0.382998
16	2022	-0.008652	0.090454	-0.095649
17	2023	-0.064127	0.097106	-0.660384
18	2024	0.004810	0.060972	0.078881

If the portfolio return is simple return.

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.043910	0.041839	1.049495
1	2007	-0.016293	0.058147	-0.280198
2	2008	0.296934	0.160720	1.847518
3	2009	0.122036	0.109193	1.117622
4	2010	0.018435	0.043345	0.425312
5	2011	0.003980	0.061090	0.065150
6	2012	-0.117483	0.046930	-2.503379
7	2013	0.021604	0.037323	0.578846
8	2014	-0.032765	0.046276	-0.708043
9	2015	-0.085004	0.054475	-1.560440
10	2016	0.063244	0.057462	1.100615
11	2017	-0.027662	0.042904	-0.644751
12	2018	-0.085255	0.055834	-1.526929
13	2019	0.002718	0.062722	0.043329
14	2020	0.290889	0.208331	1.396285
15	2021	-0.031919	0.087163	-0.366196
16	2022	0.011964	0.090675	0.131940
17	2023	0.062317	0.118982	0.523748
18	2024	0.009616	0.060877	0.157958

Comments:

Compared to the result in part C, the portfolio's total annual return and sharpe ratio improves, showing the benefits of controlling the turnover rate.

The strategy shows robustness and a suitable turnover rate will improve the performance. When gamma=1, the portfolio annual return and sharpe ratio are lower than that of gamma = 1.5 most of time, indicating that the high turnover rate will erode return due to trading cost. Gamma = 1.5 or gamma = 2 shows no obvious difference on performance before 2012, but after 2012, portfolio with gamma = 2 shows a better performance.

partA

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
pd.set_option('display.max_rows', 100)
```

In [3]:
```
df_price = pd.read_csv('data_export/data_us/adjusted.csv',parse_dates=['Date'])
df_tickers = pd.read_csv('data_export/data_us/tickers.csv',names=['ticker','GISC code'])
df_univ = pd.read_csv('data_export/data_us/univ_h.csv')
```

In [4]:
```
df_tickers['industry'] = df_tickers.iloc[:,1].astype(str).str[:6]
df_tickers = df_tickers.dropna(subset=['GISC code'])
#drop goog and nws
df_tickers.drop(df_tickers[(df_tickers['ticker']=='GOOG') | (df_tickers['ticker']=='NWS')].index,inplace=True)
#drop '#N/A Invalid Security' row
df_tickers = df_tickers[df_tickers['GISC code'] != '#N/A Invalid Security']
df_tickers
```

Out[4]:

	ticker	GISC code	industry
9	0910150D	25301020	253010
19	1284849D	35202010	352020
20	1288453D	30201030	302010
27	1448062D	50201030	502010
28	1500785D	55103010	551030
...	...	...	...
959	YUM	25301040	253010
960	ZBH	35101010	351010
961	ZBRA	45203010	452030
962	ZION	40101015	401010
963	ZTS	35202010	352020

781 rows × 3 columns

In [5]:
```
df_price = df_price.set_index('Date')
df_log_return = np.log(df_price / df_price.shift(1)).replace([np.inf, -np.inf], np.nan)

df_log_return = df_log_return.iloc[1:]
df_log_return = df_log_return.fillna(0)
df_log_return
```

Out[5]:

Date	0111145D	0202445Q	0203524D	0226226D	0544749D	0574018D	0772031D	0848680D	0867887D	0910150D	...	XOM	XRA
2004-01-05	0.005072	0.001627	0.036087	-0.017489	0.018884	0.000720	0.071045	0.040209	0.0	0.009351	...	0.023112	-0.0167
2004-01-06	0.001685	-0.013090	0.017325	0.026073	0.023216	-0.000720	-0.004425	0.039217	0.0	0.001161	...	-0.006754	0.0015
2004-01-07	-0.010152	0.020725	-0.015079	-0.004761	-0.001425	-0.008668	-0.013423	-0.013092	0.0	-0.018177	...	-0.007293	-0.0052
2004-01-08	-0.006826	0.004599	0.045047	-0.053103	-0.001832	-0.000726	0.017848	0.014223	0.0	0.012935	...	-0.002443	0.0063
2004-01-09	0.005975	-0.020629	-0.008367	0.023077	0.008320	-0.000727	-0.029178	0.000000	0.0	-0.011157	...	-0.015024	-0.0018
...	...	...	...	...	...	...	...	...	...	...	...	...	
2025-12-24	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	...	-0.001676	0.0125
2025-12-26	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	...	-0.000923	0.0407
2025-12-29	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	...	0.011851	-0.0237
2025-12-30	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	...	0.003809	0.0132
2025-12-31	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	...	-0.005387	0.0026

5534 rows × 964 columns

In [6]:
```
#calculate 5-day log return
df_ret_5d = np.log(df_price / df_price.shift(5)).replace([np.inf, -np.inf], np.nan)
df_ret_5d = df_ret_5d.iloc[5:]
#fillna with 0
df_ret_5d = df_ret_5d.fillna(0)
df_ret_5d.head(10)
```

Out[6]:

Date	0111145D	0202445Q	0203524D	0226226D	0544749D	0574018D	0772031D	0848680D	0867887D	0910150D	...	XOM	XRA
2004-01-09	-0.004246	-0.006769	0.075014	-0.026203	0.047163	-0.010121	0.041867	0.080557	0.0	-0.005888	...	-0.008401	-0.0157
2004-01-12	-0.024324	0.008786	0.073146	-0.021578	0.023621	-0.011568	-0.031454	0.068209	0.0	-0.015829	...	-0.017468	-0.0105
2004-01-13	-0.042999	0.015178	0.023278	-0.063616	0.001625	-0.021081	-0.057153	0.044255	0.0	-0.012861	...	-0.015131	-0.0218
2004-01-14	-0.020185	0.016458	0.015079	-0.048385	0.001223	-0.008016	-0.029712	0.078762	0.0	0.007078	...	-0.007838	-0.0224
2004-01-15	-0.013793	0.020661	-0.019960	0.009814	-0.013123	-0.015369	-0.068678	0.049601	0.0	-0.013526	...	-0.015275	-0.0287
2004-01-16	-0.019769	0.023149	0.001677	-0.002954	-0.005063	-0.024648	-0.030055	0.059760	0.0	-0.010091	...	0.005198	-0.0211
2004-01-20	-0.007806	-0.002074	-0.049465	-0.004192	0.020497	-0.022805	-0.016106	0.021740	0.0	-0.010097	...	0.000490	-0.0178
2004-01-21	0.018717	0.044200	-0.050133	0.077105	0.027988	-0.014805	0.054933	0.011835	0.0	0.007035	...	0.018272	-0.0011
2004-01-22	0.008209	0.020198	-0.071431	0.084554	0.021301	-0.018456	0.000000	-0.020332	0.0	-0.002356	...	0.014646	0.0104
2004-01-23	0.021894	0.014501	-0.081179	0.081941	0.046586	-0.008146	0.025738	-0.001076	0.0	-0.000591	...	0.018693	0.0116

10 rows × 964 columns

In [7]:
```
#compute 21-day rolling volatility
df_rolling_vol = df_log_return.rolling(window=21,min_periods=5).std()
```

```
# df_rolling_vol = df_rolling_vol.loc[df_ret_5d.index]
df_rolling_vol = df_rolling_vol[4:]
#set the floor of volatility to 0.003
df_rolling_vol = df_rolling_vol.clip(lower=0.003)
df_rolling_vol.head(10)
```

Out[7]:

Date	0111145D	0202445Q	0203524D	0226226D	0544749D	0574018D	0772031D	0848680D	0867887D	0910150D	...	XOM	XRAY
2004-01-09	0.007251	0.016133	0.026475	0.032494	0.011460	0.003766	0.038944	0.023614	0.003	0.013267	...	0.014582	0.008704
2004-01-12	0.008687	0.016293	0.024946	0.029230	0.011754	0.003410	0.035103	0.021659	0.003	0.011869	...	0.014537	0.008487
2004-01-13	0.009488	0.015211	0.029774	0.026921	0.010957	0.004453	0.034921	0.019800	0.003	0.011012	...	0.013424	0.007990
2004-01-14	0.010814	0.015999	0.030105	0.025753	0.010538	0.004884	0.032639	0.018379	0.003	0.010222	...	0.012428	0.007400
2004-01-15	0.010150	0.015081	0.028183	0.024351	0.012171	0.004987	0.031567	0.020424	0.003	0.009892	...	0.012097	0.007148
2004-01-16	0.009607	0.015826	0.026644	0.023424	0.012246	0.005231	0.029903	0.019304	0.003	0.009571	...	0.011584	0.007524
2004-01-20	0.009116	0.015295	0.026342	0.022476	0.012660	0.005152	0.028544	0.019712	0.003	0.009085	...	0.011369	0.007264
2004-01-21	0.009356	0.018391	0.027482	0.029310	0.012102	0.004918	0.029428	0.018887	0.003	0.010873	...	0.011453	0.007632
2004-01-22	0.009013	0.017687	0.029338	0.028414	0.012264	0.004822	0.030961	0.019089	0.003	0.010645	...	0.011062	0.007719
2004-01-23	0.009480	0.016993	0.028190	0.027300	0.011837	0.004816	0.029756	0.018394	0.003	0.010340	...	0.010794	0.007479

10 rows × 964 columns

In [8]:
```
print(df_ret_5d.shape)
print(df_rolling_vol.shape)

print(df_ret_5d.index.equals(df_rolling_vol.index))
print(df_ret_5d.columns.equals(df_rolling_vol.columns))
```

```
(5530, 964)
(5530, 964)
True
True
```

In [9]:
```
df_ret_5d_nml = df_ret_5d.div(df_rolling_vol)
df_ret_5d_nml.head(10)
```

Out[9]:

Date	0111145D	0202445Q	0203524D	0226226D	0544749D	0574018D	0772031D	0848680D	0867887D	0910150D	...	XOM	XRA
2004-01-09	-0.585591	-0.419557	2.833384	-0.806387	4.115403	-2.687431	1.075052	3.411362	0.0	-0.443773	...	-0.576115	-1.8146
2004-01-12	-2.799994	0.539250	2.932203	-0.738223	2.009530	-3.392609	-0.896053	3.149199	0.0	-1.333625	...	-1.201640	-1.2401
2004-01-13	-4.531957	0.997822	0.781839	-2.363033	0.148285	-4.734329	-1.636638	2.235085	0.0	-1.167866	...	-1.127141	-2.7318
2004-01-14	-1.866634	1.028651	0.500885	-1.878779	0.116031	-1.641112	-0.910296	4.285358	0.0	0.692438	...	-0.630654	-3.0291
2004-01-15	-1.358948	1.370006	-0.708238	0.403014	-1.078268	-3.082105	-2.175635	2.428585	0.0	-1.367419	...	-1.262677	-4.0276
2004-01-16	-2.057804	1.462705	0.062935	-0.126091	-0.413459	-4.711691	-1.005105	3.095821	0.0	-1.054334	...	0.448701	-2.8101
2004-01-20	-0.856298	-0.135597	-1.877832	-0.186527	1.618970	-4.426226	-0.564250	1.102879	0.0	-1.111410	...	0.043058	-2.4607
2004-01-21	2.000587	2.403300	-1.824226	2.630681	2.312684	-3.010153	1.866704	0.626616	0.0	0.646989	...	1.595395	-0.1525
2004-01-22	0.910799	1.142008	-2.434758	2.975779	1.736908	-3.827826	0.000000	-1.065109	0.0	-0.221304	...	1.324052	1.3552
2004-01-23	2.309524	0.853353	-2.879662	3.001527	3.935570	-1.691315	0.864945	-0.058522	0.0	-0.057203	...	1.731858	1.5533

10 rows × 964 columns

In [ ]:
```
# extract ticker list of each year from df_univ
df_univ = df_univ.set_index('year')
univ_dict = {
    year: df_univ.columns[df_univ.loc[year] == 1].tolist()
    for year in df_univ.index
}
```

In [ ]:
```
# create a mask with same size as df_ret_5d_nml
df_ret_5d_nml.columns = df_ret_5d_nml.columns.str.strip()

mask = pd.DataFrame(
    False,
    index=df_ret_5d_nml.index,
    columns=df_ret_5d_nml.columns
)

for year, tickers in univ_dict.items():
    year_rows = df_ret_5d_nml.index.year == year
    valid_cols = [t for t in tickers if t in df_ret_5d_nml.columns]
    mask.loc[year_rows, valid_cols] = True

# apply the mask on df_ret_5d_nml, assign np.nan to values not in the universe
df_ret_filtered = df_ret_5d_nml.where(mask,np.nan)
df_ret_filtered
```

Out[ ]:

Date	0111145D	0202445Q	0203524D	0226226D	0544749D	0574018D	0772031D	0848680D	0867887D	0910150D	...	XOM	XRAY
2004-01-09	-0.585591	-0.419557	2.833384	-0.806387	4.115403	NaN	1.075052	3.411362	NaN	-0.443773	...	-0.576115	NaN
2004-01-12	-2.799994	0.539250	2.932203	-0.738223	2.009530	NaN	-0.896053	3.149199	NaN	-1.333625	...	-1.201640	NaN
2004-01-13	-4.531957	0.997822	0.781839	-2.363033	0.148285	NaN	-1.636638	2.235085	NaN	-1.167866	...	-1.127141	NaN
2004-01-14	-1.866634	1.028651	0.500885	-1.878779	0.116031	NaN	-0.910296	4.285358	NaN	0.692438	...	-0.630654	NaN
2004-01-15	-1.358948	1.370006	-0.708238	0.403014	-1.078268	NaN	-2.175635	2.428585	NaN	-1.367419	...	-1.262677	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2025-12-24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.231002	NaN
2025-12-26	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.814036	NaN
2025-12-29	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	2.649493	NaN
2025-12-30	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.962851	NaN
2025-12-31	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	0.630714	NaN

5530 rows × 964 columns

In [ ]:
```
#convert to long format
df_5d_ret_long = (
    df_ret_filtered
    .stack(dropna=False)
    .reset_index()
)

df_5d_ret_long.columns = ['date', 'ticker', '5d_ret']
df_5d_ret_long
```

Out[ ]:

	date	ticker	5d_ret
0	2004-01-09	0111145D	-0.585591
1	2004-01-09	0202445Q	-0.419557
2	2004-01-09	0203524D	2.833384
3	2004-01-09	0226226D	-0.806387
4	2004-01-09	0544749D	4.115403
...	...	...	...
5330915	2025-12-31	YUM	-1.620455
5330916	2025-12-31	ZBH	0.858266
5330917	2025-12-31	ZBRA	-1.236315
5330918	2025-12-31	ZION	NaN
5330919	2025-12-31	ZTS	1.389272

5330920 rows × 3 columns

In [ ]:
```
#merge with industry code
df_5d_ret_long = df_5d_ret_long.merge(df_tickers[['ticker', 'industry']], on='ticker', how='left')
df_5d_ret_long
```

Out[ ]:

	date	ticker	5d_ret	industry
0	2004-01-09	0111145D	-0.585591	NaN
1	2004-01-09	0202445Q	-0.419557	NaN
2	2004-01-09	0203524D	2.833384	NaN
3	2004-01-09	0226226D	-0.806387	NaN
4	2004-01-09	0544749D	4.115403	NaN
...	...	...	...	...
5330915	2025-12-31	YUM	-1.620455	253010
5330916	2025-12-31	ZBH	0.858266	351010
5330917	2025-12-31	ZBRA	-1.236315	452030
5330918	2025-12-31	ZION	NaN	401010
5330919	2025-12-31	ZTS	1.389272	352020

5330920 rows × 4 columns

In [ ]:
```
#calculate industry average for variable (prior 5-day return)
df_industry_5d_ret = (
    df_5d_ret_long
    .dropna(subset=['industry'])
    .groupby(['date', 'industry'])['5d_ret']
    .mean()
    .reset_index()
)
df_industry_5d_ret.rename(columns={'5d_ret': 'industry_5d_ret'}, inplace=True)
df_industry_5d_ret
```

Out[ ]:

	date	industry	industry_5d_ret
0	2004-01-09	101010	2.357957
1	2004-01-09	101020	1.951389
2	2004-01-09	151010	-0.523583
3	2004-01-09	151020	1.368388
4	2004-01-09	151030	0.315965
...	...	...	...
387095	2025-12-31	601050	0.100130
387096	2025-12-31	601060	1.582779
387097	2025-12-31	601070	-0.006499
387098	2025-12-31	601080	0.921591
387099	2025-12-31	602010	-0.536621

387100 rows × 3 columns

In [ ]:
```
# merge df_df_df_industry_ret and df_5d_ret_long on industry
df_5d_ret_long = df_5d_ret_long.merge(df_industry_5d_ret, on=['date', 'industry'], how='left')
df_5d_ret_long
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret
0	2004-01-09	0111145D	-0.585591	NaN	NaN
1	2004-01-09	0202445Q	-0.419557	NaN	NaN
2	2004-01-09	0203524D	2.833384	NaN	NaN
3	2004-01-09	0226226D	-0.806387	NaN	NaN
4	2004-01-09	0544749D	4.115403	NaN	NaN
...	...	...	...	...	...
5330915	2025-12-31	YUM	-1.620455	253010	-1.299014
5330916	2025-12-31	ZBH	0.858266	351010	-0.619273
5330917	2025-12-31	ZBRA	-1.236315	452030	-1.092482
5330918	2025-12-31	ZION	NaN	401010	-1.567176
5330919	2025-12-31	ZTS	1.389272	352020	0.704949

5330920 rows × 5 columns

In [ ]:
```
# demean the variable by industry mean
df_5d_ret_long['5d_ret_minus_industry'] = df_5d_ret_long['5d_ret'] - df_5d_ret_long['industry_5d_ret']
df_5d_ret_long
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry
0	2004-01-09	0111145D	-0.585591	NaN	NaN	NaN
1	2004-01-09	0202445Q	-0.419557	NaN	NaN	NaN
2	2004-01-09	0203524D	2.833384	NaN	NaN	NaN
3	2004-01-09	0226226D	-0.806387	NaN	NaN	NaN
4	2004-01-09	0544749D	4.115403	NaN	NaN	NaN
...	...	...	...	...	...	...
5330915	2025-12-31	YUM	-1.620455	253010	-1.299014	-0.321441
5330916	2025-12-31	ZBH	0.858266	351010	-0.619273	1.477540
5330917	2025-12-31	ZBRA	-1.236315	452030	-1.092482	-0.143833
5330918	2025-12-31	ZION	NaN	401010	-1.567176	NaN
5330919	2025-12-31	ZTS	1.389272	352020	0.704949	0.684322

5330920 rows × 6 columns

In [ ]:
```
# create a mask with same size as df_log_return
df_log_return.columns = df_log_return.columns.str.strip()

mask = pd.DataFrame(
    False,
    index=df_log_return.index,
    columns=df_log_return.columns
)

for year, tickers in univ_dict.items():
    year_rows = df_log_return.index.year == year
    valid_cols = [t for t in tickers if t in df_log_return.columns]
    mask.loc[year_rows, valid_cols] = True

# apply mask on df_log_return, assign np.nan to values not in the universe
df_log_return_filtered = df_log_return.where(mask,np.nan)

# convert daily return into long format
df_log_return_long = (
    df_log_return_filtered
    .stack(dropna=False)
    .reset_index()
)

df_log_return_long.columns = ['date', 'ticker', 'log_return']

# merge daily return with industry
df_log_return_long = df_log_return_long.merge(df_tickers[['ticker', 'industry']], on='ticker', how='left')

# calculate the industry average return
```

```
df_industry_ret = (
    df_log_return_long
    .dropna(subset=['industry'])
    .groupby(['date', 'industry'])['log_return']
    .mean()
    .reset_index()
)
df_industry_ret.rename(columns={'log_return': 'industry_log_return'}, inplace=True)

# merge df_industry_ret and df_log_return_long on industry
df_log_return_long = df_log_return_long.merge(df_industry_ret, on=['date', 'industry'], how='left')

# demean the daily return by industry mean
df_log_return_long['log_ret_minus_industry'] = df_log_return_long['log_return'] - df_log_return_long['industry_log_return']
df_log_return_long
```

Out[ ]:

	date	ticker	log_return	industry	industry_log_return	log_ret_minus_industry
0	2004-01-05	0111145D	0.005072	NaN	NaN	NaN
1	2004-01-05	0202445Q	0.001627	NaN	NaN	NaN
2	2004-01-05	0203524D	0.036087	NaN	NaN	NaN
3	2004-01-05	0226226D	-0.017489	NaN	NaN	NaN
4	2004-01-05	0544749D	0.018884	NaN	NaN	NaN
...	...	...	...	...	...	...
5334771	2025-12-31	YUM	-0.005932	253010	-0.007299	0.001368
5334772	2025-12-31	ZBH	-0.009629	351010	-0.008030	-0.001599
5334773	2025-12-31	ZBRA	-0.016015	452030	-0.013802	-0.002212
5334774	2025-12-31	ZION	NaN	401010	-0.007451	NaN
5334775	2025-12-31	ZTS	-0.004678	352020	-0.004652	-0.000026

5334776 rows × 6 columns

In [ ]:

```
# merge df_log_return_long and df_5d_ret_long on ticker and date
df_final = df_5d_ret_long.merge(df_log_return_long[['date', 'ticker','log_return', 'log_ret_minus_industry']], on=['date',
df_final.sort_values(by=['date', 'ticker'], inplace=True)
df_final
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry	log_return	log_ret_minus_industry
0	2004-01-09	0111145D	-0.585591	NaN	NaN	NaN	0.005975	NaN
1	2004-01-09	0202445Q	-0.419557	NaN	NaN	NaN	-0.020629	NaN
2	2004-01-09	0203524D	2.833384	NaN	NaN	NaN	-0.008367	NaN
3	2004-01-09	0226226D	-0.806387	NaN	NaN	NaN	0.023077	NaN
4	2004-01-09	0544749D	4.115403	NaN	NaN	NaN	0.008320	NaN
...	...	...	...	...	...	...	...	...
5330915	2025-12-31	YUM	-1.620455	253010	-1.299014	-0.321441	-0.005932	0.001368
5330916	2025-12-31	ZBH	0.858266	351010	-0.619273	1.477540	-0.009629	-0.001599
5330917	2025-12-31	ZBRA	-1.236315	452030	-1.092482	-0.143833	-0.016015	-0.002212
5330918	2025-12-31	ZION	NaN	401010	-1.567176	NaN	NaN	NaN
5330919	2025-12-31	ZTS	1.389272	352020	0.704949	0.684322	-0.004678	-0.000026

5330920 rows × 8 columns

In [19]:

```
df_final['log_ret_minus_industry_plus_1'] = (
    df_final
    .groupby('ticker')['log_ret_minus_industry']
    .shift(-1)
)
df_final
```

Out[19]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry	log_return	log_ret_minus_industry	log_ret_minus_industr
0	2004-01-09	0111145D	-0.585591	NaN	NaN	NaN	0.005975	NaN	
1	2004-01-09	0202445Q	-0.419557	NaN	NaN	NaN	-0.020629	NaN	
2	2004-01-09	0203524D	2.833384	NaN	NaN	NaN	-0.008367	NaN	
3	2004-01-09	0226226D	-0.806387	NaN	NaN	NaN	0.023077	NaN	
4	2004-01-09	0544749D	4.115403	NaN	NaN	NaN	0.008320	NaN	
...	...	...	...	...	...	...	...	...	
5330915	2025-12-31	YUM	-1.620455	253010	-1.299014	-0.321441	-0.005932	0.001368	
5330916	2025-12-31	ZBH	0.858266	351010	-0.619273	1.477540	-0.009629	-0.001599	
5330917	2025-12-31	ZBRA	-1.236315	452030	-1.092482	-0.143833	-0.016015	-0.002212	
5330918	2025-12-31	ZION	NaN	401010	-1.567176	NaN	NaN	NaN	
5330919	2025-12-31	ZTS	1.389272	352020	0.704949	0.684322	-0.004678	-0.000026	

5330920 rows × 9 columns

In [ ]:
```
# calculate beta
df_reg = df_final.dropna(subset=['log_ret_minus_industry_plus_1', '5d_ret_minus_industry'])

beta_series = (
    df_reg
    .groupby('date')
    .apply(lambda x:
        (x['log_ret_minus_industry_plus_1'] * x['5d_ret_minus_industry']).sum()
        /
        (x['5d_ret_minus_industry']**2).sum() if (x['5d_ret_minus_industry']**2).sum() != 0 else np.nan
    )
)
beta_series
```

Out[ ]:
```
date
2004-01-09   -0.000159
2004-01-12    0.000360
2004-01-13   -0.000059
2004-01-14   -0.000300
2004-01-15    0.000835
                ...
2025-12-23   -0.000533
2025-12-24    0.000018
2025-12-26   -0.000353
2025-12-29    0.000240
2025-12-30    0.000826
Length: 5529, dtype: float64
```

In [ ]:
```
# covert beta series into beta dataframe
beta_series = beta_series.reset_index()
beta_series.columns = ['date', 'beta']
beta_series
```

Out[ ]:

	date	beta
0	2004-01-09	-0.000159
1	2004-01-12	0.000360
2	2004-01-13	-0.000059
3	2004-01-14	-0.000300
4	2004-01-15	0.000835
...	...	...
5524	2025-12-23	-0.000533
5525	2025-12-24	0.000018
5526	2025-12-26	-0.000353
5527	2025-12-29	0.000240
5528	2025-12-30	0.000826

5529 rows × 2 columns

In [ ]:
```
# calculate mean, std, number, t-stats for beta annually
beta_series['year'] = beta_series['date'].dt.year
yearly_stats = (
    beta_series
    .groupby('year')['beta']
    .agg(['mean', 'std', 'count'])
    .reset_index()
)
yearly_stats['t_stat'] = (yearly_stats['mean'] / yearly_stats['std']) * np.sqrt(yearly_stats['count'])
yearly_stats
```

Out[ ]:

	year	mean	std	count	t_stat
0	2004	-0.000136	0.000624	247	-3.427817
1	2005	-0.000133	0.000549	252	-3.845856
2	2006	-0.000160	0.000576	251	-4.401218
3	2007	-0.000081	0.000901	251	-1.422194
4	2008	-0.000497	0.003370	253	-2.348262
5	2009	-0.000137	0.002006	252	-1.086134
6	2010	-0.000057	0.000763	252	-1.195891
7	2011	-0.000176	0.001293	252	-2.164281
8	2012	0.000022	0.000676	250	0.511471
9	2013	-0.000065	0.000562	252	-1.842169
10	2014	-0.000025	0.000642	252	-0.624152
11	2015	-0.000005	0.000893	252	-0.097222
12	2016	-0.000070	0.001048	252	-1.055238
13	2017	-0.000043	0.000593	251	-1.155244
14	2018	0.000028	0.000935	251	0.472556
15	2019	0.000090	0.001160	252	1.227006
16	2020	-0.000375	0.003804	253	-1.568538
17	2021	0.000015	0.001479	252	0.161380
18	2022	-0.000013	0.001790	251	-0.119097
19	2023	-0.000173	0.001740	250	-1.575897
20	2024	-0.000104	0.000878	252	-1.883139
21	2025	0.000003	0.001411	249	0.037566

partB

In [ ]:
```
# calculate beta bar
beta_bar = {}
for y in range(2006, 2025): #2006-2024
    start = max(2004, y - 5)
```

```
    end = y - 1

    window = beta_series.loc[
        (beta_series['year'] >= start) & (beta_series['year'] <= end),
        'beta'
    ]

    beta_bar[y] = window.mean()   # 直接对“日度beta”求均值（自动忽略NaN）

df_beta_bar = pd.Series(beta_bar, name='beta_bar').reset_index()
df_beta_bar.columns = ['year', 'beta_bar']
df_beta_bar
```

Out[ ]:

	year	beta_bar
0	2006	-1.345812e-04
1	2007	-1.430699e-04
2	2008	-1.274825e-04
3	2009	-2.021267e-04
4	2010	-2.020834e-04
5	2011	-1.869645e-04
6	2012	-1.902097e-04
7	2013	-1.698888e-04
8	2014	-8.305165e-05
9	2015	-6.060993e-05
10	2016	-5.019070e-05
11	2017	-2.882728e-05
12	2018	-4.176072e-05
13	2019	-2.315456e-05
14	2020	-1.418075e-07
15	2021	-7.442227e-05
16	2022	-5.747505e-05
17	2023	-5.154340e-05
18	2024	-9.162150e-05

In [ ]:
```
# merge df_final and df_beta_bar on year
df_final['year'] = df_final['date'].dt.year
df_final_beta = pd.merge(df_final, df_beta_bar, on='year', how='left')
df_final_beta.dropna(subset=['beta_bar'], inplace=True)

df_final_beta
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry	log_return	log_ret_minus_industry	log_ret_minus_industr
481036	2006-01-03	0111145D	1.820895	NaN	NaN	NaN	0.044080	NaN	
481037	2006-01-03	0202445Q	0.604735	NaN	NaN	NaN	0.020088	NaN	
481038	2006-01-03	0203524D	-0.293967	NaN	NaN	NaN	0.036657	NaN	
481039	2006-01-03	0226226D	2.451989	NaN	NaN	NaN	0.024321	NaN	
481040	2006-01-03	0544749D	1.405995	NaN	NaN	NaN	0.036948	NaN	
...	...	...	...	...	...	...	...	...	
5089915	2024-12-31	YUM	0.224810	253010	-0.476562	0.701373	0.004781	0.003385	
5089916	2024-12-31	ZBH	-0.974400	351010	-0.590139	-0.384261	0.002179	0.000853	
5089917	2024-12-31	ZBRA	-0.768009	452030	-0.700073	-0.067937	0.006155	0.005099	
5089918	2024-12-31	ZION	-0.144845	401010	-0.086945	-0.057900	-0.001288	-0.001367	
5089919	2024-12-31	ZTS	-0.722500	352020	-0.635544	-0.086955	0.004244	-0.000836	

4608884 rows × 11 columns

In [ ]:
```
# calculate expected return
df_final_beta['expected ret'] = df_final_beta['beta_bar'] * df_final_beta['5d_ret_minus_industry']
df_final_beta
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry	log_return	log_ret_minus_industry	log_ret_minus_industr
481036	2006-01-03	0111145D	1.820895	NaN	NaN	NaN	0.044080	NaN	
481037	2006-01-03	0202445Q	0.604735	NaN	NaN	NaN	0.020088	NaN	
481038	2006-01-03	0203524D	-0.293967	NaN	NaN	NaN	0.036657	NaN	
481039	2006-01-03	0226226D	2.451989	NaN	NaN	NaN	0.024321	NaN	
481040	2006-01-03	0544749D	1.405995	NaN	NaN	NaN	0.036948	NaN	
...	...	...	...	...	...	...	...	...	
5089915	2024-12-31	YUM	0.224810	253010	-0.476562	0.701373	0.004781	0.003385	
5089916	2024-12-31	ZBH	-0.974400	351010	-0.590139	-0.384261	0.002179	0.000853	
5089917	2024-12-31	ZBRA	-0.768009	452030	-0.700073	-0.067937	0.006155	0.005099	
5089918	2024-12-31	ZION	-0.144845	401010	-0.086945	-0.057900	-0.001288	-0.001367	
5089919	2024-12-31	ZTS	-0.722500	352020	-0.635544	-0.086955	0.004244	-0.000836	

4608884 rows × 12 columns

In [ ]:
```
# get signal
df_signal = df_final_beta.dropna(subset=['expected ret']).copy()
# calculate daily rank
df_signal['pct_rank'] = (
    df_signal
    .groupby('date')['expected ret']
```

```
    .rank(pct=True)
)
# create signal based on daily rank
df_signal['signal'] = 0

df_signal.loc[df_signal['pct_rank'] >= 0.8, 'signal'] = 1
df_signal.loc[df_signal['pct_rank'] <= 0.2, 'signal'] = -1

# convert log return to simple return
df_signal['simple return'] = np.exp(df_signal['log_return']) - 1

df_signal
```

Out[ ]:

	date	ticker	5d_ret	industry	industry_5d_ret	5d_ret_minus_industry	log_return	log_ret_minus_industry	log_ret_minus_industr
481045	2006-01-03	0910150D	3.599892	253010	0.138429	3.461463	0.014821	0.008875	-
481055	2006-01-03	1284849D	0.047181	352020	-0.043515	0.090696	0.010686	-0.004091	
481064	2006-01-03	1500785D	0.736812	551030	0.247299	0.489512	0.020165	0.003391	
481065	2006-01-03	1518855D	0.422717	502030	0.391362	0.031355	0.043208	0.018917	
481066	2006-01-03	1519128D	-0.456631	451030	-0.147437	-0.309193	0.038272	0.021842	-
...	...	...	...	...	...	...	...	...	
5089915	2024-12-31	YUM	0.224810	253010	-0.476562	0.701373	0.004781	0.003385	
5089916	2024-12-31	ZBH	-0.974400	351010	-0.590139	-0.384261	0.002179	0.000853	-
5089917	2024-12-31	ZBRA	-0.768009	452030	-0.700073	-0.067937	0.006155	0.005099	
5089918	2024-12-31	ZION	-0.144845	401010	-0.086945	-0.057900	-0.001288	-0.001367	
5089919	2024-12-31	ZTS	-0.722500	352020	-0.635544	-0.086955	0.004244	-0.000836	-

2248599 rows × 15 columns

In [ ]:
```
# calculate portfolio return
df_signal['signal_plus_1'] = df_signal.groupby('ticker')['signal'].shift(1) #signal向下移动一行
df_signal.dropna(subset=['signal_plus_1'], inplace=True)
```

In [ ]:
```
def calc_port_ret(x):
    long_mask = (x['signal_plus_1'] == 1) & x['log_return'].notna()
    short_mask = (x['signal_plus_1'] == -1) & x['log_return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})

    long_sum = x.loc[long_mask, 'log_return'].sum()
    short_sum = x.loc[short_mask, 'log_return'].sum()

    port_ret = (long_sum - short_sum) / Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port = (
    df_signal.groupby('date', as_index=False)
      .apply(calc_port_ret)
      .reset_index(drop=True)
)

daily_port
```

Out[ ]:

	date	Nl	Ns	port_ret
0	2006-01-04	74.0	73.0	0.003024
1	2006-01-05	74.0	73.0	0.001250
2	2006-01-06	74.0	73.0	0.002781
3	2006-01-09	74.0	73.0	-0.003618
4	2006-01-10	74.0	73.0	0.001711
...	...	...	...	...
4775	2024-12-24	101.0	100.0	0.001868
4776	2024-12-26	101.0	100.0	-0.000422
4777	2024-12-27	101.0	100.0	0.001046
4778	2024-12-30	101.0	100.0	-0.001506
4779	2024-12-31	101.0	100.0	0.000015

4780 rows × 4 columns

In [ ]:
```
# calculate total annual return and annual return volatility for portfolio
daily_port['year'] = daily_port['date'].dt.year
annual_stats = (
    daily_port
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.190455	0.041244	4.617770
1	2007	0.088001	0.065845	1.336490
2	2008	0.391532	0.174598	2.242474
3	2009	0.109670	0.112984	0.970663
4	2010	0.055799	0.045738	1.219981
5	2011	0.130569	0.062524	2.088287
6	2012	0.024490	0.048039	0.509802
7	2013	0.040373	0.037840	1.066937
8	2014	0.033948	0.042939	0.790602
9	2015	0.018476	0.054238	0.340644
10	2016	0.029181	0.065333	0.446653
11	2017	0.071588	0.042162	1.697940
12	2018	-0.052182	0.052882	-0.986755
13	2019	-0.045562	0.069855	-0.652238
14	2020	0.305517	0.227956	1.340246
15	2021	0.030800	0.097973	0.314375
16	2022	0.091288	0.105011	0.869325
17	2023	0.086310	0.101511	0.850247
18	2024	0.129462	0.065771	1.968373

In [ ]:
```
#2. portfolio return based on simple return
def calc_port_ret_simple(x):
    long_mask = (x['signal_plus_1'] == 1) & x['simple return'].notna()
    short_mask = (x['signal_plus_1'] == -1) & x['simple return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
```

```
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})

    long_sum = x.loc[long_mask, 'simple return'].sum()
    short_sum = x.loc[short_mask, 'simple return'].sum()

    port_ret = (long_sum - short_sum) / Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port_simple = (
    df_signal.groupby('date', as_index=False)
      .apply(calc_port_ret_simple)
      .reset_index(drop=True)
)

# calculate total annual return and annual return volatility for portfolio
daily_port_simple['year'] = daily_port_simple['date'].dt.year
annual_stats = (
    daily_port_simple
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.193411	0.041480	4.662778
1	2007	0.095204	0.065689	1.449316
2	2008	0.489273	0.177393	2.758131
3	2009	0.116485	0.113923	1.022486
4	2010	0.057587	0.046140	1.248106
5	2011	0.144049	0.062353	2.310227
6	2012	0.031025	0.048323	0.642034
7	2013	0.044356	0.037989	1.167620
8	2014	0.037012	0.042798	0.864800
9	2015	0.026472	0.054244	0.488024
10	2016	0.033925	0.065404	0.518700
11	2017	0.077183	0.042068	1.834703
12	2018	-0.045347	0.052663	-0.861086
13	2019	-0.032402	0.069320	-0.467427
14	2020	0.359413	0.223838	1.605686
15	2021	0.031828	0.098009	0.324747
16	2022	0.110289	0.104589	1.054503
17	2023	0.190540	0.119254	1.597760
18	2024	0.128885	0.065405	1.970569

Part C

In [ ]:
```
#trading cost
COST_PER_TRADE = 0.0005

df_signal['pos'] = df_signal['signal_plus_1']
df_signal['pos_next'] = df_signal['signal']
df_signal['trade'] = (df_signal['pos'] - df_signal['pos_next']).abs()
```

In [ ]:
```
#1. portfolio return based onlog return
def calc_port_ret_cost(x):
    long_mask = (x['signal_plus_1'] == 1) & x['log_return'].notna()
    short_mask = (x['signal_plus_1'] == -1) & x['log_return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})
```

```
    long_sum = x.loc[long_mask, 'log_return'].sum()
    short_sum = x.loc[short_mask, 'log_return'].sum()
    trade_sum = x['trade'].sum()

    port_ret = (long_sum - short_sum) / Nl - trade_sum*COST_PER_TRADE/Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port_cost = (
    df_signal.groupby('date', as_index=False)
      .apply(calc_port_ret_cost)
      .reset_index(drop=True)
)

# calculate total annual return and annual return volatility for portfolio
daily_port_cost['year'] = daily_port_cost['date'].dt.year
annual_stats = (
    daily_port_cost
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.015117	0.040844	0.370118
1	2007	-0.084480	0.065430	-1.291155
2	2008	0.212849	0.174141	1.222284
3	2009	-0.063418	0.112534	-0.563547
4	2010	-0.115639	0.045462	-2.543630
5	2011	-0.044533	0.062145	-0.716598
6	2012	-0.148896	0.047615	-3.127101
7	2013	-0.132584	0.037505	-3.535121
8	2014	-0.139809	0.042672	-3.276374
9	2015	-0.153106	0.053791	-2.846284
10	2016	-0.141978	0.064888	-2.188044
11	2017	-0.098731	0.041832	-2.360203
12	2018	-0.220858	0.052333	-4.220228
13	2019	-0.216470	0.069259	-3.125514
14	2020	0.127672	0.227349	0.561568
15	2021	-0.139785	0.097511	-1.433526
16	2022	-0.078906	0.104451	-0.755435
17	2023	-0.081888	0.101090	-0.810053
18	2024	-0.042569	0.065441	-0.650487

In [ ]:
```
#2. portfolio return based on simple return
def calc_port_ret_simple_cost(x):
    long_mask = (x['signal_plus_1'] == 1) & x['simple return'].notna()
    short_mask = (x['signal_plus_1'] == -1) & x['simple return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})

    long_sum = x.loc[long_mask, 'simple return'].sum()
    short_sum = x.loc[short_mask, 'simple return'].sum()
    trade_sum = x['trade'].sum()

    port_ret = (long_sum - short_sum) / Nl -trade_sum*COST_PER_TRADE/Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port_simple_cost = (
    df_signal.groupby('date', as_index=False)
```

```
      .apply(calc_port_ret_simple_cost)
      .reset_index(drop=True)
)

# calculate total annual return and annual return volatility for portfolio
daily_port_simple_cost['year'] = daily_port_simple_cost['date'].dt.year
annual_stats = (
    daily_port_simple_cost
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.018073	0.041080	0.439945
1	2007	-0.077277	0.065279	-1.183796
2	2008	0.310591	0.176938	1.755364
3	2009	-0.056603	0.113471	-0.498836
4	2010	-0.113851	0.045862	-2.482475
5	2011	-0.031053	0.061972	-0.501077
6	2012	-0.142361	0.047900	-2.972048
7	2013	-0.128601	0.037650	-3.415696
8	2014	-0.136745	0.042532	-3.215097
9	2015	-0.145109	0.053802	-2.697080
10	2016	-0.137235	0.064962	-2.112547
11	2017	-0.093136	0.041735	-2.231594
12	2018	-0.214023	0.052112	-4.106996
13	2019	-0.203310	0.068721	-2.958471
14	2020	0.181569	0.223230	0.813371
15	2021	-0.138757	0.097550	-1.422418
16	2022	-0.059905	0.104028	-0.575852
17	2023	0.022342	0.118902	0.187902
18	2024	-0.043146	0.065071	-0.663062

Part D

In [34]:
```
df_signal.drop(columns=['industry','5d_ret','industry_5d_ret','year'],inplace=True)
```

In [35]:
```
#normalize expected return
df_signal['w0'] = (
    df_signal.groupby('date')['expected ret']
      .transform(lambda x: 0.5 * np.abs(x).sum())
)

df_signal['normalize ret'] = df_signal['expected ret'] / df_signal['w0']
df_signal.drop(columns='w0',inplace=True)
df_signal
```

Out[35]:

	date	ticker	5d_ret_minus_industry	log_return	log_ret_minus_industry	log_ret_minus_industry_plus_1	beta_bar	expected ret	pct_ra
482009	2006-01-04	0910150D	1.758916	-0.009856	-0.016252	0.004931	-0.000135	-0.000237	0.1089
482019	2006-01-04	1284849D	1.042392	0.010756	0.003570	-0.014611	-0.000135	-0.000140	0.2316
482028	2006-01-04	1500785D	1.215280	0.004551	0.006726	-0.005217	-0.000135	-0.000164	0.1907
482029	2006-01-04	1518855D	0.863916	0.001466	0.011197	-0.010333	-0.000135	-0.000116	0.2806
482030	2006-01-04	1519128D	0.147146	0.002142	-0.007787	0.005766	-0.000135	-0.000020	0.4550
...	...	...	...	...	...	...	...	...	
5089915	2024-12-31	YUM	0.701373	0.004781	0.003385	0.001233	-0.000092	-0.000064	0.1477
5089916	2024-12-31	ZBH	-0.384261	0.002179	0.000853	-0.007764	-0.000092	0.000035	0.7225
5089917	2024-12-31	ZBRA	-0.067937	0.006155	0.005099	0.004754	-0.000092	0.000006	0.5628
5089918	2024-12-31	ZION	-0.057900	-0.001288	-0.001367	NaN	-0.000092	0.000005	0.5608
5089919	2024-12-31	ZTS	-0.086955	0.004244	-0.000836	-0.002430	-0.000092	0.000008	0.5668

2247838 rows × 16 columns

In [59]:
```
df_d = df_signal[['date','ticker','log_return','simple return','pos','normalize ret']].copy()
df_d
```

Out[59]:

	date	ticker	log_return	simple return	pos	normalize ret
482009	2006-01-04	0910150D	-0.009856	-0.009808	-1.0	-0.008337
482019	2006-01-04	1284849D	0.010756	0.010814	0.0	-0.004941
482028	2006-01-04	1500785D	0.004551	0.004561	0.0	-0.005760
482029	2006-01-04	1518855D	0.001466	0.001467	0.0	-0.004095
482030	2006-01-04	1519128D	0.002142	0.002144	0.0	-0.000697
...	...	...	...	...	...	...
5089915	2024-12-31	YUM	0.004781	0.004793	-1.0	-0.005134
5089916	2024-12-31	ZBH	0.002179	0.002182	1.0	0.002813
5089917	2024-12-31	ZBRA	0.006155	0.006174	1.0	0.000497
5089918	2024-12-31	ZION	-0.001288	-0.001287	0.0	0.000424
5089919	2024-12-31	ZTS	0.004244	0.004253	1.0	0.000637

2247838 rows × 6 columns

In [ ]:
```
gamma = 2
dates = df_d['date'].drop_duplicates().tolist()

# daily position list
out_list = []

# ticker -> p in {-1,0,1}
df_target_date = df_d[df_d['date'] == '2006-01-04']
pos_prev = dict(zip(df_target_date['ticker'], df_target_date['pos'])) # pos at t, calculated by previous signal
```

In [ ]:
```
for dt in dates:
    d = df_d[df_d['date'] == dt].copy()

    #  universe (ticker with normalize ret)
    tickers = d['ticker'].tolist()
    N = len(tickers)

    # sigma and hurdle
```

```
    sigma = d['normalize ret'].std()
    h = gamma * sigma

    # append today's position to today's df
    d['p_prev'] = d['ticker'].map(pos_prev).fillna(0).astype(int)

    # construct x and y
    # x: long score
    d['x'] = d['normalize ret']
    d.loc[d['p_prev'] == 0,  'x'] = d.loc[d['p_prev'] == 0,  'normalize ret'] - h
    d.loc[d['p_prev'] == -1, 'x'] = d.loc[d['p_prev'] == -1, 'normalize ret'] - 2*h

    # y: short score
    d['y'] = -d['normalize ret']
    d.loc[d['p_prev'] == 1,  'y'] = -d.loc[d['p_prev'] == 1,  'normalize ret'] - 2*h
    d.loc[d['p_prev'] == 0,  'y'] = -d.loc[d['p_prev'] == 0,  'normalize ret'] - h

    # number of ticker to long
    K = int(np.floor(0.2 * N))
    K = max(K, 1)

    # long: x, the largest K number of return
    long_names = d.nlargest(K, 'x')['ticker'].tolist()

    # short: y, the largest K number of return, avoid overlapping with x
    d_short_pool = d[~d['ticker'].isin(long_names)].copy()
    short_names = d_short_pool.nlargest(K, 'y')['ticker'].tolist()

    # new position for tomorrow
    d['pos'] = 0
    d.loc[d['ticker'].isin(long_names), 'pos'] = 1
    d.loc[d['ticker'].isin(short_names), 'pos'] = -1

    out_list.append(d[['date','ticker','p_prev','pos']])

    # update pos_prev for the next day
    for tkr, pnew in zip(d['ticker'], d['pos']):
        pos_prev[tkr] = int(pnew)

df_pos = pd.concat(out_list, ignore_index=True)
df_pos
```

Out[ ]:

	date	ticker	p_prev	pos
0	2006-01-04	0910150D	-1	-1
1	2006-01-04	1284849D	0	0
2	2006-01-04	1500785D	0	0
3	2006-01-04	1518855D	0	0
4	2006-01-04	1519128D	0	0
...	...	...	...	...
2247833	2024-12-31	YUM	-1	-1
2247834	2024-12-31	ZBH	0	0
2247835	2024-12-31	ZBRA	1	1
2247836	2024-12-31	ZION	0	0
2247837	2024-12-31	ZTS	1	1

2247838 rows × 4 columns

In [62]:

```
df_pos['trade'] = (df_pos['p_prev']-df_pos['pos']).abs()
df_pos
```

Out[62]:

	date	ticker	p_prev	pos	trade
0	2006-01-04	0910150D	-1	-1	0
1	2006-01-04	1284849D	0	0	0
2	2006-01-04	1500785D	0	0	0
3	2006-01-04	1518855D	0	0	0
4	2006-01-04	1519128D	0	0	0
...	...	...	...	...	...
2247833	2024-12-31	YUM	-1	-1	0
2247834	2024-12-31	ZBH	0	0	0
2247835	2024-12-31	ZBRA	1	1	0
2247836	2024-12-31	ZION	0	0	0
2247837	2024-12-31	ZTS	1	1	0

2247838 rows × 5 columns

In [ ]:
```
df_d.drop(columns='pos',inplace=True)
df_d = df_d.merge(df_pos[['date','ticker','p_prev','trade']],on=['date','ticker'],how='left')
df_d
```

Out[ ]:

	date	ticker	log_return	simple return	normalize ret	p_prev	trade
0	2006-01-04	0910150D	-0.009856	-0.009808	-0.008337	-1	0
1	2006-01-04	1284849D	0.010756	0.010814	-0.004941	0	0
2	2006-01-04	1500785D	0.004551	0.004561	-0.005760	0	0
3	2006-01-04	1518855D	0.001466	0.001467	-0.004095	0	0
4	2006-01-04	1519128D	0.002142	0.002144	-0.000697	0	0
...	...	...	...	...	...	...	...
2247833	2024-12-31	YUM	0.004781	0.004793	-0.005134	-1	0
2247834	2024-12-31	ZBH	0.002179	0.002182	0.002813	0	0
2247835	2024-12-31	ZBRA	0.006155	0.006174	0.000497	1	0
2247836	2024-12-31	ZION	-0.001288	-0.001287	0.000424	0	0
2247837	2024-12-31	ZTS	0.004244	0.004253	0.000637	1	0

2247838 rows × 7 columns

In [ ]:
```
#calculate porfolio return based on log return
def calc_port_ret_cost(x):
    long_mask = (x['p_prev'] == 1) & x['log_return'].notna()
    short_mask = (x['p_prev'] == -1) & x['log_return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})

    long_sum = x.loc[long_mask, 'log_return'].sum()
    short_sum = x.loc[short_mask, 'log_return'].sum()
    trade_sum = x['trade'].sum()

    port_ret = (long_sum - short_sum) / Nl -trade_sum*COST_PER_TRADE/Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port_cost = (
    df_d.groupby('date', as_index=False)
      .apply(calc_port_ret_cost)
      .reset_index(drop=True)
)

# calculate total annual return and annual return volatility for portfolio
daily_port_cost['year'] = daily_port_cost['date'].dt.year
annual_stats = (
    daily_port_cost
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
```

```
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.039599	0.041696	0.949712
1	2007	-0.029032	0.058096	-0.499726
2	2008	0.214133	0.163864	1.306774
3	2009	0.095247	0.107688	0.884473
4	2010	0.012209	0.043195	0.282638
5	2011	-0.010182	0.061013	-0.166875
6	2012	-0.126863	0.047510	-2.670259
7	2013	0.016685	0.037088	0.449880
8	2014	-0.039215	0.046456	-0.844136
9	2015	-0.094673	0.054220	-1.746085
10	2016	0.053764	0.057893	0.928673
11	2017	-0.034583	0.043132	-0.801805
12	2018	-0.097141	0.056128	-1.730696
13	2019	-0.012603	0.063012	-0.200006
14	2020	0.237469	0.208137	1.140924
15	2021	-0.033398	0.087202	-0.382998
16	2022	-0.008652	0.090454	-0.095649
17	2023	-0.064127	0.097106	-0.660384
18	2024	0.004810	0.060972	0.078881

In [ ]:
```
# calculate porfolio return based on simple return
def calc_port_ret_cost(x):
    long_mask = (x['p_prev'] == 1) & x['simple return'].notna()
    short_mask = (x['p_prev'] == -1) & x['simple return'].notna()

    Nl = long_mask.sum()
    Ns = short_mask.sum()

    if Nl == 0:
        return pd.Series({'Nl': 0, 'Ns': Ns, 'port_ret': np.nan})

    long_sum = x.loc[long_mask, 'simple return'].sum()
    short_sum = x.loc[short_mask, 'simple return'].sum()
    trade_sum = x['trade'].sum()

    port_ret = (long_sum - short_sum) / Nl - trade_sum*COST_PER_TRADE/Nl
    return pd.Series({'Nl': Nl, 'Ns': Ns, 'port_ret': port_ret})

daily_port_cost = (
    df_d.groupby('date', as_index=False)
      .apply(calc_port_ret_cost)
      .reset_index(drop=True)
)

# calculate total annual return and annual return volatility
daily_port_cost['year'] = daily_port_cost['date'].dt.year
annual_stats = (
    daily_port_cost
    .groupby('year')['port_ret']
    .agg(['sum', 'std','mean','count'])
    .reset_index()
)
annual_stats.rename(columns={'sum': 'total annual return', 'std': 'return std', 'mean': 'annual mean return', 'count': 'num
annual_stats['annual return vol'] = annual_stats['return std'] * np.sqrt(annual_stats['number of trading days'])
annual_stats['sharpe_ratio'] = (annual_stats['annual mean return'] / annual_stats['return std'])*np.sqrt(annual_stats['numb
annual_stats[['year', 'total annual return', 'annual return vol', 'sharpe_ratio']]
```

Out[ ]:

	year	total annual return	annual return vol	sharpe_ratio
0	2006	0.043910	0.041839	1.049495
1	2007	-0.016293	0.058147	-0.280198
2	2008	0.296934	0.160720	1.847518
3	2009	0.122036	0.109193	1.117622
4	2010	0.018435	0.043345	0.425312
5	2011	0.003980	0.061090	0.065150
6	2012	-0.117483	0.046930	-2.503379
7	2013	0.021604	0.037323	0.578846
8	2014	-0.032765	0.046276	-0.708043
9	2015	-0.085004	0.054475	-1.560440
10	2016	0.063244	0.057462	1.100615
11	2017	-0.027662	0.042904	-0.644751
12	2018	-0.085255	0.055834	-1.526929
13	2019	0.002718	0.062722	0.043329
14	2020	0.290889	0.208331	1.396285
15	2021	-0.031919	0.087163	-0.366196
16	2022	0.011964	0.090675	0.131940
17	2023	0.062317	0.118982	0.523748
18	2024	0.009616	0.060877	0.157958